Resenha: Big Ball of Mud (Foote & Yoder)

Arquitetura acidental, forças do mundo real e estratégias para evoluir sistemas legados

Referência

FOOTE, Brian; YODER, Joseph. **Big Ball of Mud**. PLoP/EuroPLoP 1997, Technical Report WUCS-97-34. Versão de 26 jun. 1999. Publicado também em *Pattern Languages of Program Design 4*, cap. 29, 2000.

Fonte 1. Referências conferidas em 10 out. 2026.

1. Introdução

Foote e Yoder analisam um fenômeno recorrente em projetos de software: a tendência de muitos sistemas acabarem se tornando uma "Big Ball of Mud", isto é, um conjunto de código e dados com estrutura fraca, alto acoplamento e evolução baseada em remendos. Em vez de tratar esse resultado apenas como falha técnica, os autores argumentam que ele surge como resposta a pressões concretas, prazo, custo, mudança de requisitos, aprendizado incompleto do domínio e necessidade de manter o sistema em operação.

2. Síntese das ideias principais

O artigo organiza o tema por meio de padrões que descrevem desde a origem do problema até caminhos de mitigação. No padrão **Big Ball of Mud**, a estrutura não foi planejada (ou se degradou com o tempo): funcionalidades se misturam, regras ficam implícitas e mudanças pequenas podem gerar efeitos colaterais. O resultado é queda de produtividade, testes frágeis e risco elevado em cada nova entrega.

2.1 Throwaway Code (protótipo que vira produto)

Códigos feitos para serem temporários frequentemente sobrevivem. Por "funcionarem", protótipos são estendidos até virarem produção, carregando suposições não documentadas e limites ocultos que mais tarde travam a evolução.

2.2 Piecemeal Growth (crescimento aos pedaços)

A evolução por pequenos "puxadinhos" aumenta a velocidade de resposta, mas pode corroer a arquitetura. Sem momentos de consolidação (refatoração e reorganização), o crescimento incremental tende a produzir emaranhamento e duplicações.

2.3 Keep It Working (manter funcionando)

Como sistemas reais não podem parar, a disciplina é manter o software operacional: integrações frequentes, mudanças pequenas e testes automatizados para reduzir risco e evitar manutenção "heroica".

2.4 Shearing Layers (camadas por ritmo de mudança)

Partes do sistema mudam em velocidades diferentes. Separar componentes por taxa de mudança protege o núcleo mais estável e concentra flexibilidade onde a mudança é inevitável (interfaces e integrações).

2.5 Sweeping It Under the Rug (isolar a sujeira)

Quando não dá para reestruturar tudo, encapsular o legado com uma fachada/adapter reduz pontos de contato e impede que a lama contamine novas áreas, criando espaço para melhorias graduais e seguras.

2.6 Reconstruction (reconstrução)

Em casos extremos, reescrever pode ser melhor. A reconstrução deve partir de um pós-morte do sistema antigo, preservando o aprendizado e evitando repetir erros por falta de memória organizacional.

3. Aplicação imaginada dos conceitos

Em produtos que começam como MVP (por exemplo, um sistema de agendamentos), é comum o protótipo virar base de produção. Para evitar a “lama”, pode-se adotar um ciclo: entregar o mínimo, garantir testes, e refatorar logo após a entrega para reduzir acoplamento e duplicação. Na estrutura do projeto, aplicar **Shearing Layers** ajuda: regras de negócio e validações ficam no núcleo; integrações externas (calendário, banco, mensageria) ficam na infraestrutura; e UI/API permanecem como bordas. Se existir um módulo legado difícil, uma fachada limita o impacto e permite evolução por etapas.

4. Conclusão

O artigo trata a arquitetura acidental como consequência previsível de forças reais, e não como mera falta de competência. A principal lição é equilibrar entrega e sustentabilidade por meio de testes e integrações frequentes, crescimento incremental com consolidação, separação em camadas por ritmo de mudança, isolamento do legado e, quando necessário, reconstrução guiada por aprendizado.